Compiler Construction

Linearization

From HIR to LIR

Inadequacy of HIR

- No nested sequences
- Assembly is imperative: there is no “expression”
- Calling conventions
- Two Way Conditional Jumps
- Limited Number of Registers

LIR & Backend

LIR

Produce a Generic and registerless assembly code

Backend

Translation of the LIR into a correct ASM.

LIR as a subset of HIR

```
<Exp> ::= "const" int
        | "name" <Label>
        | "temp" <Temp>
        | "binop" <Oper> <Exp> <Exp>
        | "mem" <Exp>
        | "call" <Exp> [{<Exp>}] "call end"
        | "eseq" <Stm> <Exp>

<Stm> ::= "move" <Exp> <Exp>
        | "sxp" <Exp>
        | "jump" <Exp> [{<Label>}]
        | "cjump" <Relop> <Exp> <Exp> <Label> <Label>
        | "seq" [{<Stm>}] "seq end"
        | "label" <Label>
```

LIR as a subset of HIR

```
<Exp> ::= "const" int
        | "name" <Label>
        | "temp" <Temp>
        | "binop" <Oper> <Exp> <Exp>
        | "mem" <Exp>
        | "call" <Exp> [{<Exp>}] "call end"
        | "eseq" <Stm> <Exp>

<Stm> ::= "move" <Exp> <Exp>
        | "sxp" <Exp>
        | "jump" <Exp> [{<Label>}]
        | "cjump" <Relop> <Exp> <Exp> <Label> <Label>
        | "seq" [<Stm>] "seq end"
        | "label" <Label>
```

LIR as a subset of HIR

```
<Exp> ::= "const" int
        | "name" <Label>
        | "temp" <Temp>
        | "binop" <Oper> <Exp> <Exp>
        | "mem" <Exp>
        | "call" <Exp> ["temp" <Temp>] "call end"
        | "eseq" <Stm> <Exp>

<Stm> ::= "move" <Exp> <Exp>
        | "sxp" <Exp>
        | "jump" <Exp> [{<Label>}]
        | "cjump" <Relop> <Exp> <Exp> <Label> <Label>
        | "seq" [<Stm>] "seq end"
        | "label" <Label>
```

Explicit Call-by-Value Transformation

Make evaluation order and side effects explicit in the IR:

A call of the form:

$$f(e_1, e_2, \ldots, e_n)$$

Is rewritten into:

- Sequential evaluation of arguments:

$$t_1 := e_1;\ t_2 := e_2;\ \ldots;\ t_n := e_n$$

  arguments may have side effects $\rightarrow$ evaluation order matters (left-to-right)

- Followed by a call:

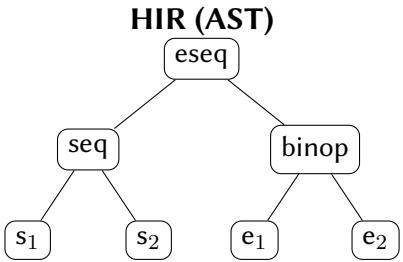
$$f(t_1, t_2, \ldots, t_n)$$

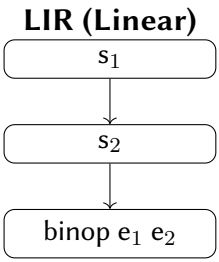
Result

Function calls become pure consumers of temporaries.

Linearization

eseq and *seq* must be eliminated





Exercise - 1

Rewrite using less `seq`:

```
seq
    seq
        s1
        s2
    seq end
    s3
seq end
```

Exercise - 1

Rewrite using less `seq`:

```
seq
    seq
        s1
        s2
    seq end
    s3
seq end
```

Rule: Sequence Flattening

Exercise - 2

Rewrite using less `eseq`:

```
eseq
      s1
      eseq
            s2
            e
```

Exercise - 2

Rewrite using less `eseq`:

```
eseq
      s1
      eseq
            s2
            e
```

Rule: Replace eseq with seqs (1)

Exercise - 3

Rewrite using less `eseq`:

```
sxp
    eseq
        s1
        e
```

Exercise - 3

Rewrite using less `eseq`:

```
sxp
      eseq
            s1
            e
```

Rule: Replace eseq with seqs (2)

Exercice - 4

Rewrite so that the binop does not contain an `eseq`:

```
binop
        add
        eseq s e1
        e2
```

Exercice - 4

Rewrite so that the binop does not contain an `eseq`:

```
binop
        add
        eseq s e1
        e2
```

Rule: Move eseq toward the root (1)

Exercice - 5

```
binop
        add
        e1
        eseq s e2
```

Exercice - 5

```
binop
        add
        e1
        eseq s e2
```

Rule: Move eseq toward the root (2)

High Level counterexample

```
let var t := 51
in
  t + (t := 42, 0)
end
```

High Level counterexample

```
let var t := 51
in
  t + (t := 42, 0)
end
```

$\rightsquigarrow$

```
let var t := 51
in
  (t := 42, t + 0)
end
```

High Level Solution

$\Rightarrow$ Save values into temporaries

High Level Solution

$\Rightarrow$ Save values into temporaries

$\rightsquigarrow$

```
let var t := 51
    var t0 := t
in
  (t := 42, t0 + 0)
end
```

Low level solution

```
binop
  add
  e1
  eseq s e2
```

Low level solution

```
binop
  add
  e1
  eseq s e2
```

$\rightsquigarrow$

```
eseq
  seq
    move temp t0 e1
    s
  seq end
  binop
    add
    temp t0
    e2
```

Linearization - Fixpoint Computation

To summarize:

1. flatten nested `seq`
2. replace `eseq` with `seq` when possible to apply 1 more often
3. move `eseq` toward root to apply 2 more often

Apply these rules until a fixpoint is reached.

Linearization - Fixpoint Computation

To summarize:

1. flatten nested `seq`
2. replace `eseq` with `seq` when possible to apply 1 more often
3. move `eseq` toward root to apply 2 more often

Apply these rules until a fixpoint is reached.

Termination

What guarantees that we will reach a fixpoint?

Termination/Correction

Because the number of Seq/Eseq always decreases

Termination/Correction

Because the number of Seq/Eseq always decreases

WRONG!!
sometimes we have to add Seqs!

Termination/Correction

Because the number of Seq/Eseq always decreases

WRONG!!
sometimes we have to add Seqs!

Depth Decreases with Each Transformation:

- Flattening `Seq` expressions (e.g. `Seq(Seq(s1, s2), s3)` $\rightsquigarrow$ `Seq(s1, s2, s3)`) reduces the nesting level by **at least 1**.

Termination/Correction

Because the number of Seq/Eseq always decreases

WRONG!!
sometimes we have to add Seqs!

Depth Decreases with Each Transformation:

- Flattening `Seq` expressions (e.g. `Seq(Seq(s1, s2), s3)` $\rightsquigarrow$ `Seq(s1, s2, s3)`) reduces the nesting level by **at least 1**.

The depth of the AST is finite, hence linearization terminates (and is correct)

Naive approach

Warning

When “de-expressioning” fresh temporaries are needed

- Detecting side-effects statically is hard.
- We can not save systematically every sub expression into temporaries!
- $\Rightarrow$ This is extremely inefficient when not needed

Exploit commutativity

Save useless extra temporaries and moves

Problem

Commutativity cannot be known statically!

E.g., `move (mem (t1), e)` and `mem (t2)` commute iff $\texttt{t1} \neq \texttt{t2}$.

Exploit commutativity

Save useless extra temporaries and moves

Problem

Commutativity cannot be known statically!

E.g., `move (mem (t1), e)` and `mem (t2)` commute iff $t1 \neq t2$.

WRONG ! because `e` can have side-effects and modify either `t1` or `t2`

Conservative Approximation

Never say “commute” when they don’t !

- “if `e` is a `const` then `s` and `e` definitely commute”.
- Otherwise, better be safe and consider they don’t commute

This what Appel implements in the book but is it possible to be more precise?

Commutativity Detection

Commutativity:
Two expressions e_1, e_2 commute if both evaluation order $e_1; e_2$ and $e_2; e_1$ are equivalent (i.e. can be swapped without affecting the outcome)

Conditions:

- **IO Equivalence**: No interference in input-output operations between expressions.
- **Memory Equivalence**: No read-write or write-read conflicts between expressions.

IO Equivalence

IO Equivalence: Two expressions can be swapped if:

- At most one performs input/output operations.

Commutative Examples:

- `2, print("Result")` — one has no IO.

Non-commutative Examples:

- `print("A"), print("B")` — order affects output.
- `x := read(), y:=read()` — order affects memory.

Memory Equivalence

Memory Equivalence: Two expressions can be swapped if:

- No memory cells are written in one and read or written in the other.
- Read-read conflicts do not matter.

Commutative Examples:

- `2, a` — no shared memory access.
- `a, (b+a)` — shared memory with no side-effects.
- `a, (b:=a)` — `b` is updated but `a` is read-only.

Non-commutative Example:

`a, (a:=2; 42)` — `a` is both read and written.

Exercise - Full Linearization

```
seq
  t <- 1
  sxp
    binop add
      eseq
        u <- binop mul temp t const 2
        temp u
      eseq
        sxp
          call name f temp t call end
        temp t
seq end
```

Exercise - Full Linearization

```
seq
  t <- 1
  sxp
    binop add
      eseq
        u <- binop mul temp t const 2
        temp u
      eseq
        sxp
          call name f temp t call end
        temp t
seq end
```

binop eseq s e1 e2 → eseq s binop e1 e2

Exercise - Full Linearization

```
seq
  t <- 1
  sxp
    eseq
      u <- binop mul temp t const 2
      binop add
        temp u
        eseq
          sxp
            call name f temp t call end
          temp t
seq end
```

Exercise - Full Linearization

```
seq
  t <- 1
  sxp
    eseq
      u <- binop mul temp t const 2
      binop add
        temp u
        eseq
          sxp
            call name f temp t call end
          temp t
seq end
```

sxp eseq s e → seq s sxp e

Exercise - Full Linearization

```
seq
  t <- 1
  seq
    u <- binop mul temp t const 2
    sxp
      binop add
        temp u
        eseq
          sxp call name f temp t call end
          temp t
   seq end
seq end
```

Exercise - Full Linearization

```
seq
  t <- 1
  seq
    u <- binop mul temp t const 2
    sxp
      binop add
        temp u
        eseq
          sxp call name f temp t call end
          temp t
   seq end
seq end
```

nested seq → flatten

Exercise - Full Linearization

```
seq
  t <- 1
  u <- binop mul temp t const 2
  sxp
    binop add
      temp u
      eseq
        sxp call name f temp t call end
        temp t
seq end
```

Exercise - Full Linearization

```
seq
  t <- 1
  u <- binop mul temp t const 2
  sxp
    binop add
      temp u
      eseq
        sxp call name f temp t call end
        temp t
seq end
```

when `s` and `e1` commute, `binop e1 eseq s e2` → `eseq s binop e1 e2`

Exercise - Full Linearization

```
seq
  t <- 1
  u <- binop mul temp t const 2
  sxp
    eseq
      sxp call name f temp t call end
      binop add
        temp u
        temp t
seq end
```

Exercise - Full Linearization

```
seq
  t <- 1
  u <- binop mul temp t const 2
  sxp
    eseq
      sxp call name f temp t call end
      binop add
        temp u
        temp t
seq end
```

WRONG!!

It's possible that f overwrites u !!!!

Exercise - Full Linearization

```
seq
  t <- 1
  u <- binop mul temp t const 2
  sxp
    eseq
      seq
        u0 <- u
        sxp call name f temp t call end
      seq end
      binop add
        temp u0
        temp t
seq end
```

Exercise - Full Linearization

```
seq
  t <- 1
  u <- binop mul temp t const 2
  sxp
    eseq
      seq
        u0 <- u
        sxp call name f temp t call end
      seq end
      binop add
        temp u0
        temp t
seq end
```

sxp eseq s e → seq s sxp e

Exercise - Full Linearization

```
seq
  t <- 1
  u <- binop mul temp t const 2
  seq
    seq
      u0 <- u
      sxp call name f temp t call end
    seq end
    sxp
      binop add
        temp u0
        temp t
seq end
```

Exercise - Full Linearization

```
seq
  t <- 1
  u <- binop mul temp t const 2
  seq
    seq
      u0 <- u
      sxp call name f temp t call end
    seq end
    sxp
      binop add
        temp u0
        temp t
seq end
```

nested seq → flatten

Exercise - Full Linearization

```
seq
  t <- 1
  u <- binop mul temp t const 2
  seq
    u0 <- u
    sxp call name f temp t call end
    sxp
      binop add
        temp u0
        temp t
seq end
```

Exercise - Full Linearization

```
seq
  t <- 1
  u <- binop mul temp t const 2
  seq
    u0 <- u
    sxp call name f temp t call end
    sxp
      binop add
        temp u0
        temp t
seq end
```

nested seq → flatten

Exercise - Full Linearization

```
seq
  t <- 1
  u <- binop mul temp t const 2
  u0 <- u
  sxp call name f temp t call end
  sxp
    binop add
      temp u0
      temp t
seq end
```

Exercise - Full Linearization

```
seq
  t <- 1
  u <- binop mul temp t const 2
  u0 <- u
  sxp call name f temp t call end
  sxp
    binop add
      temp u0
      temp t
seq end
```

We have reached a fixed point !

Summary